

Espace de recherche et algorithmes gloutons

Partie II

1. Espace de recherche

1.1 Recherche complète

Une méthode qui s'applique peu importe la discipline, c'est d'identifier l'*espace des candidats* pour la solution d'un problème.

1.1 Recherche complète

Une méthode qui s'applique peu importe la discipline, c'est d'identifier l'*espace des candidats* pour la solution d'un problème.

Si on nomme A l'ensemble des candidats pour la solution d'un problème, il faut chercher à réduire sa taille pour essayer de passer en revue chaque solution.

Une manière de le réduire est de s'appuyer sur :

- Les contraintes des variables définies par le problème.
- Les contraintes de ce qui est recherché dans le problème.

1.1 Recherche complète

Une méthode qui s'applique peu importe la discipline, c'est d'identifier l'*espace des candidats* pour la solution d'un problème.

Si on nomme A l'ensemble des candidats pour la solution d'un problème, il faut chercher à réduire sa taille pour essayer de passer en revue chaque solution.

Une manière de le réduire est de s'appuyer sur :

- Les contraintes des variables définies par le problème.
- Les contraintes de ce qui est recherché dans le problème.

Ensuite il est intéressant en général lorsque l'espace est petit, de faire une recherche complète (càd du **brute-forcing**) sur l'ensemble de candidats.

1.2 Astuce de gestion de l'espace

Une méthode incontournable lorsqu'on cherche une solution, c'est de trier les valeurs selon un certain critère lorsque cela s'avère nécessaire.

Nous n'allons pas voir les différents types de tri ici, mais seulement comment trier les éléments en Python :

```
# Les valeurs sont toujours triées dans l'ordre croissant en Python
a = [1, 12, 8, 11, 9, 0, 1]
a.sort() # a = [0, 1, 1, 8, 9, 11, 12]
b = sorted(a) # Même fonctionnement que sort, sauf qu'elle renvoie une copie
# Ici avec key, on définit une fonction qui va donner la valeur de
# comparaison pour chaque élément v.
a.sort(key=lambda v : -v) # a = [12, 11, 9, 8, 1, 1, 0]
```

2. Stratégie gloutonne *greedy*

2.1 Stratégie gloutonne

Definition 2.1.1

Lorsqu'on doit construire une solution étape par étape, le principe de la **stratégie gloutonne** (en anglais *greedy*), c'est à chaque étape de faire le choix qui semble le plus optimal à ce moment là.

Le fait qu'on construise petit à petit la solution, cela la rend très efficace.

2.1 Stratégie gloutonne

Definition 2.1.1

Lorsqu'on doit construire une solution étape par étape, le principe de la **stratégie gloutonne** (en anglais *greedy*), c'est à chaque étape de faire le choix qui semble le plus optimal à ce moment là.

Le fait qu'on construise petit à petit la solution, cela la rend très efficace.

Exemple classique. On souhaite donner le nombre minimal de pièces (en euro) qu'il faut pour former une somme d'argent.

2.1 Stratégie gloutonne

Definition 2.1.1

Lorsqu'on doit construire une solution étape par étape, le principe de la **stratégie gloutonne** (en anglais *greedy*), c'est à chaque étape de faire le choix qui semble le plus optimal à ce moment là.

Le fait qu'on construise petit à petit la solution, cela la rend très efficace.

Exemple classique. On souhaite donner le nombre minimal de pièces (en euro) qu'il faut pour former une somme d'argent.

La stratégie gloutonne ici c'est prendre chacune des pièces dans l'ordre décroissant, et d'utiliser au maximum la pièce pour réduire la somme. Lorsqu'elle atteint 0, le résultat c'est la somme des pièces utilisées.

2.2 Inconvénients de cette stratégie

Ainsi, la stratégie gloutonne aura tendance à suivre ce schéma :

1. Initialiser une solution vide.
2. Tant que la solution n'est pas complète : Sélectionner l'élément qui semble le plus prometteur selon un critère local. Ajouter cet élément à la solution courante si cela reste possible.
3. Retourner la solution obtenue.

2.2 Inconvénients de cette stratégie

Ainsi, la stratégie gloutonne aura tendance à suivre ce schéma :

1. Initialiser une solution vide.
2. Tant que la solution n'est pas complète : Sélectionner l'élément qui semble le plus prometteur selon un critère local. Ajouter cet élément à la solution courante si cela reste possible.
3. Retourner la solution obtenue.



Important

Faire des choix localement optimaux, n'implique pas que la solution globale sera optimale.

2.2 Inconvénients de cette stratégie

Ainsi, la stratégie gloutonne aura tendance à suivre ce schéma :

1. Initialiser une solution vide.
2. Tant que la solution n'est pas complète : Sélectionner l'élément qui semble le plus prometteur selon un critère local. Ajouter cet élément à la solution courante si cela reste possible.
3. Retourner la solution obtenue.

Important

Faire des choix localement optimaux, n'implique pas que la solution globale sera optimale.

C'est ici que la stratégie peut être piégeuse, et pour savoir si votre stratégie est en fin de compte globalement optimale, vous pouvez :

- Démontrer que c'est le cas
- Trouver un contre-exemple (choix très judicieux)

Note

Il est aussi possible qu'il n'existe tout simplement pas de solution gloutonne.

3. Crédits

Auteurs et images : Bilel MAHDJOUR et Patrick BIZOT

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

4. Sources

- [USACO guide - Introduction to greedy algorithms](#)
- [CP algorithms](#)
- [CPH - greedy algorithms](#)